

# **Lecture 4**

## **Browser Server (FTP and HTTP)**



# Chapter 27

## WWW and HTTP

## 27-1 ARCHITECTURE

*The **WWW** today is a distributed client/server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called sites.*

### **Topics discussed in this section:**

**Client (Browser)**

**Server**

**Uniform Resource Locator**

**Cookies**

**The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server.**

**Each site holds one or more documents, (*Web pages*).**

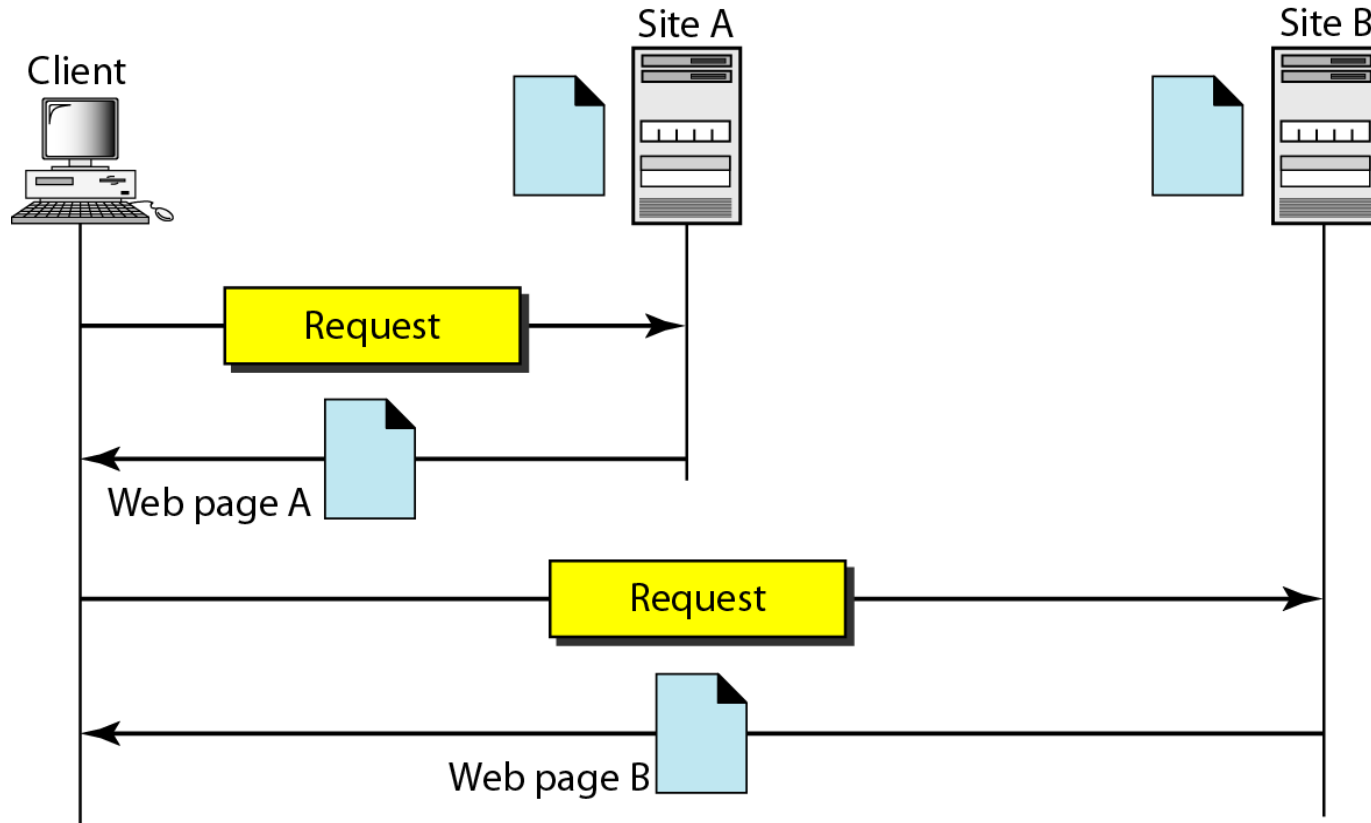
***Each Web page* can contain a link to other pages in the same site or at other sites.**

**The pages can be retrieved and viewed by using browsers.**

---

**Figure 27.1** *Architecture of WWW*

---



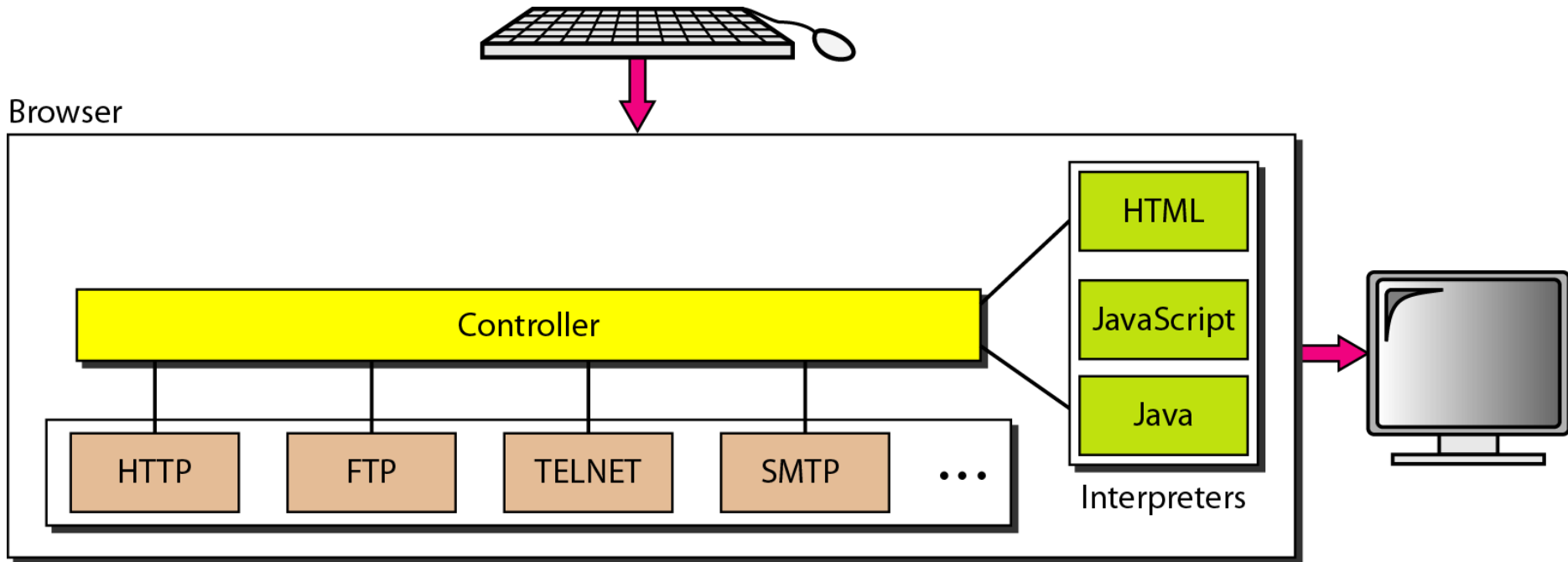
## Client (Browser)

- ❑ A variety of vendors offer commercial browsers that interpret and display a Web document, and all use nearly the same architecture.
- ❑ browser usually consists of three parts:  
a **controller**, **client protocol**, and **interpreters**.

# Client (Browser)

- The controller receives input from the keyboard or the mouse and uses the client protocol to access the document.
- After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.
- The client protocol can be one of the protocols described previously such as FTP or HTTP.
- The interpreter can be HTML, Java, or JavaScript

**Figure 27.2** *Browser*



# server

- The Web page is stored at the server. Each time a client request arrives, the corresponding document is sent to the client.
- To improve efficiency, servers normally store requested files in a cache in memory;
- A server can also become more efficient through multithreading or multiprocessing. In this case, a server can answer more than one request at a time.

# Uniform Resource Locator

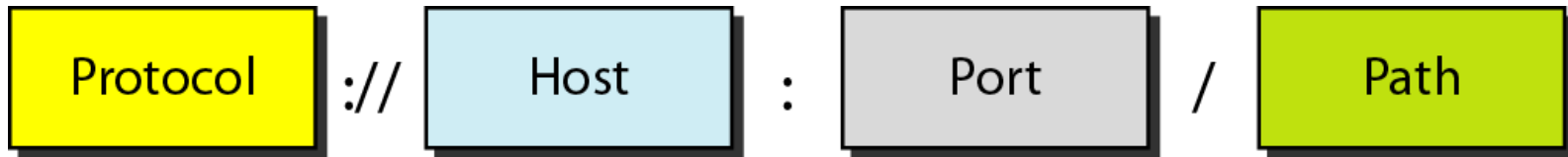
■ The uniform resource locator (URL) is a standard for specifying any kind of information on the Internet.

■ The URL defines four things: protocol, host computer, port, and path

---

**Figure 27.3** *URL*

---



- **The protocol** is the client/server program used to retrieve the document
- **The host** is the computer on which the information is located.
- The URL can optionally contain the port number of the server. usually 80 but many on 8080
- **Path** is the pathname of the file where the information is located

## 27-2 WEB DOCUMENTS

*The documents in the WWW can be grouped into three broad categories: **static**, **dynamic**, and **active**. The category is based on the time at which the contents of the document are determined.*

### **Topics discussed in this section:**

**Static Documents**

**Dynamic Documents**

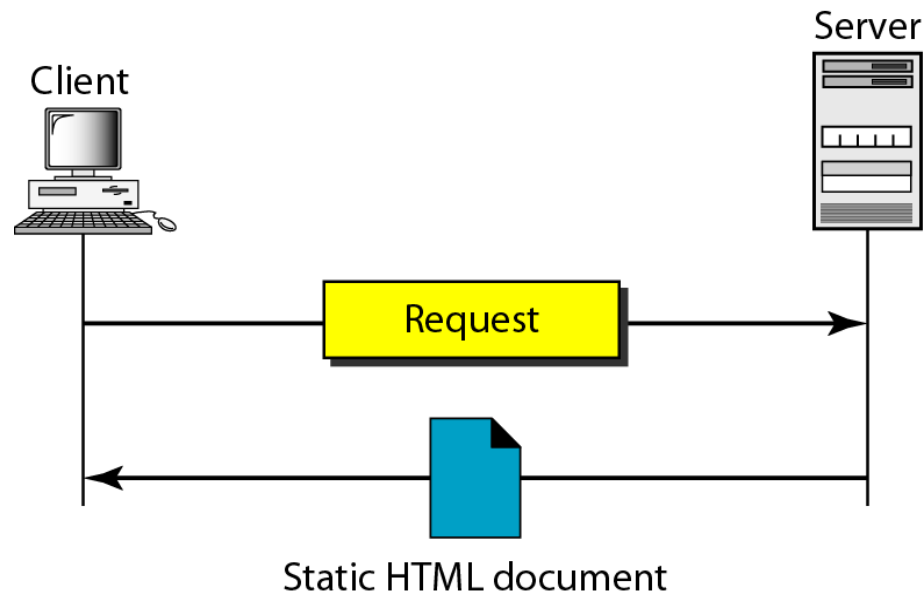
**Active Documents**

---

**Figure 27.4** *Static document*

---

Static documents are fixed-content documents that are created and stored in a server. The client can get only a copy of the document.



---

## Figure 27.5 *Boldface tags*

---

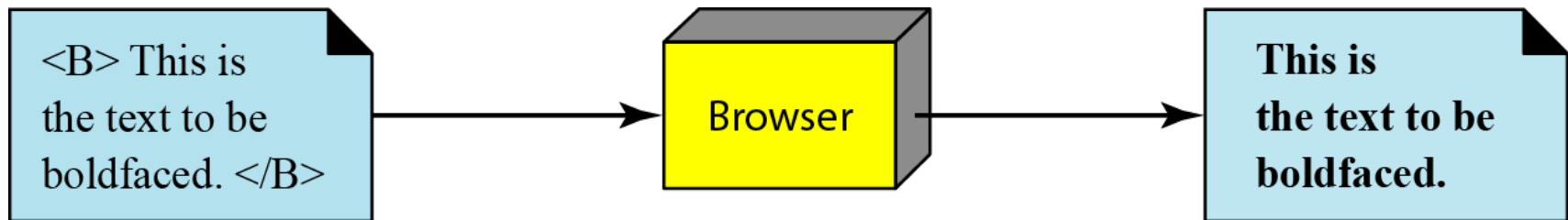
Hypertext Markup Language (HTML) is a language for creating Web pages. The term *markup language* comes from the book publishing industry. Before a book is typeset and printed, a copy editor reads the manuscript and puts marks on it. These marks tell the compositor how to format the text.



---

**Figure 27.6** *Effect of boldface tags*

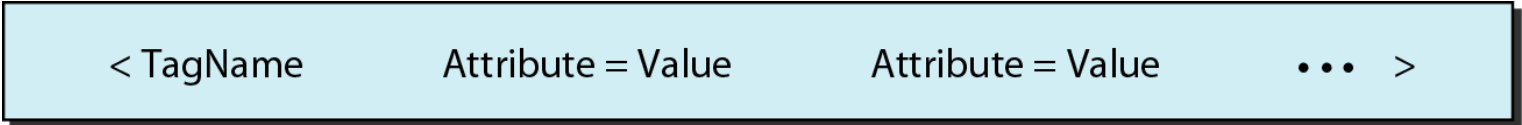
---



---

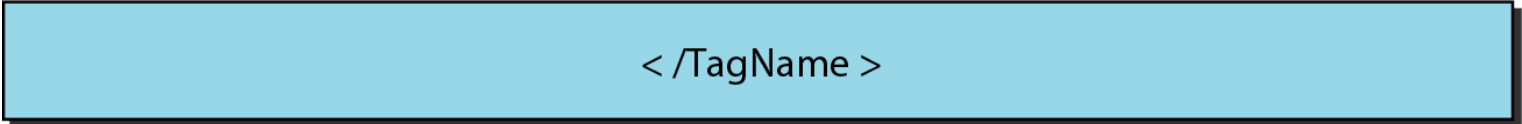
## Figure 27.7 *Beginning and ending tags*

---



A light blue rectangular box with a black border containing the text: `< TagName      Attribute = Value      Attribute = Value      ... >`

a. Beginning tag



A light blue rectangular box with a black border containing the text: `< /TagName >`

b. Ending tag

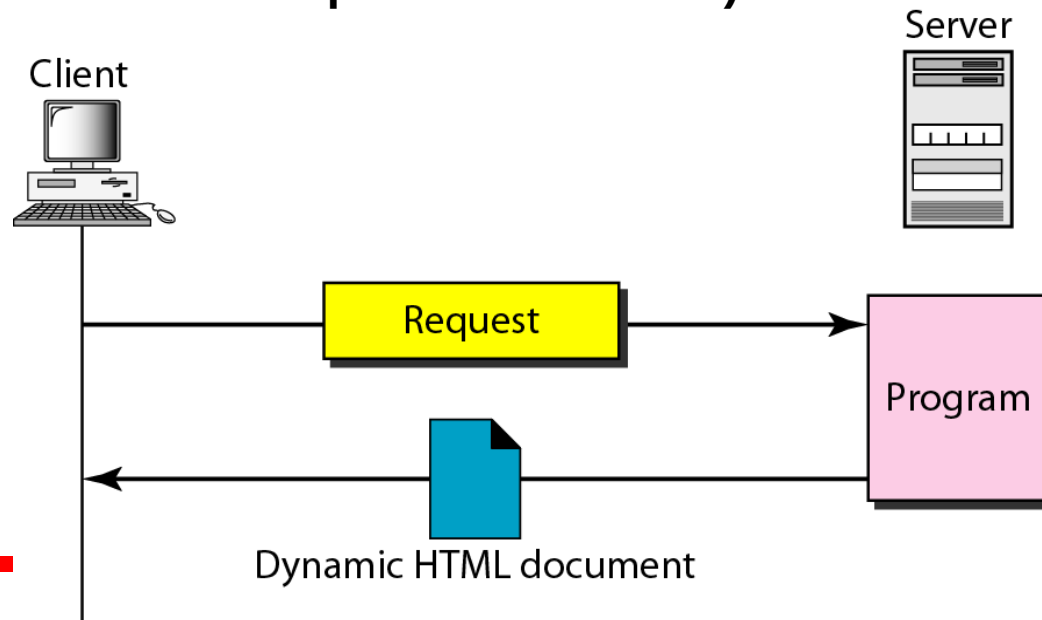
---

## Figure 27.8 *Dynamic document using CGI*

---

A dynamic document is created by a Web server whenever a browser requests the document. When a request arrives, the Web server runs an application program or a script that creates the dynamic document. The server returns the output of the program or script as a response to the browser that requested the document.

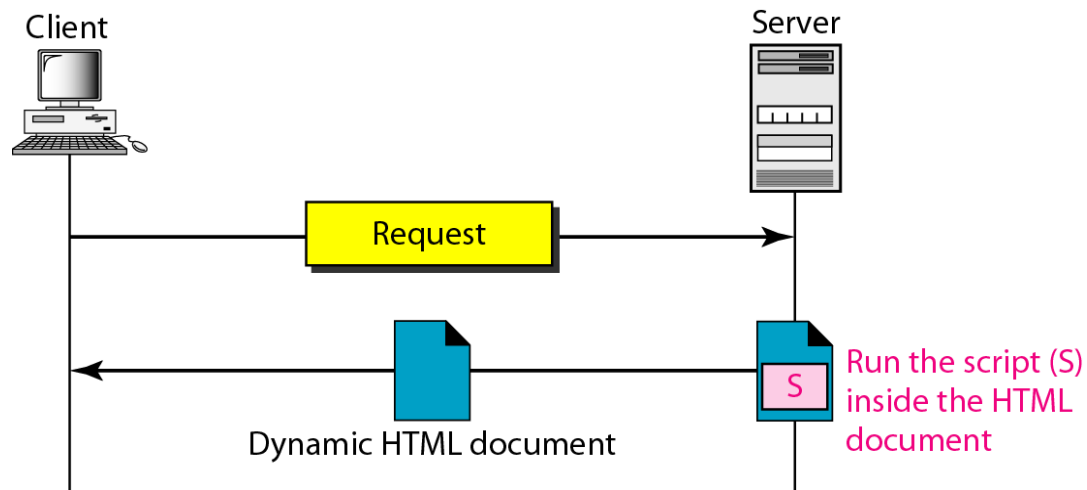
**The Common Gateway Interface (CGI)** is a technology that creates and handles dynamic documents.(how a dynamic document is written, how data are input to the program, and how the output result is used)



---

**Figure 27.9** *Dynamic document using server-site script*

---





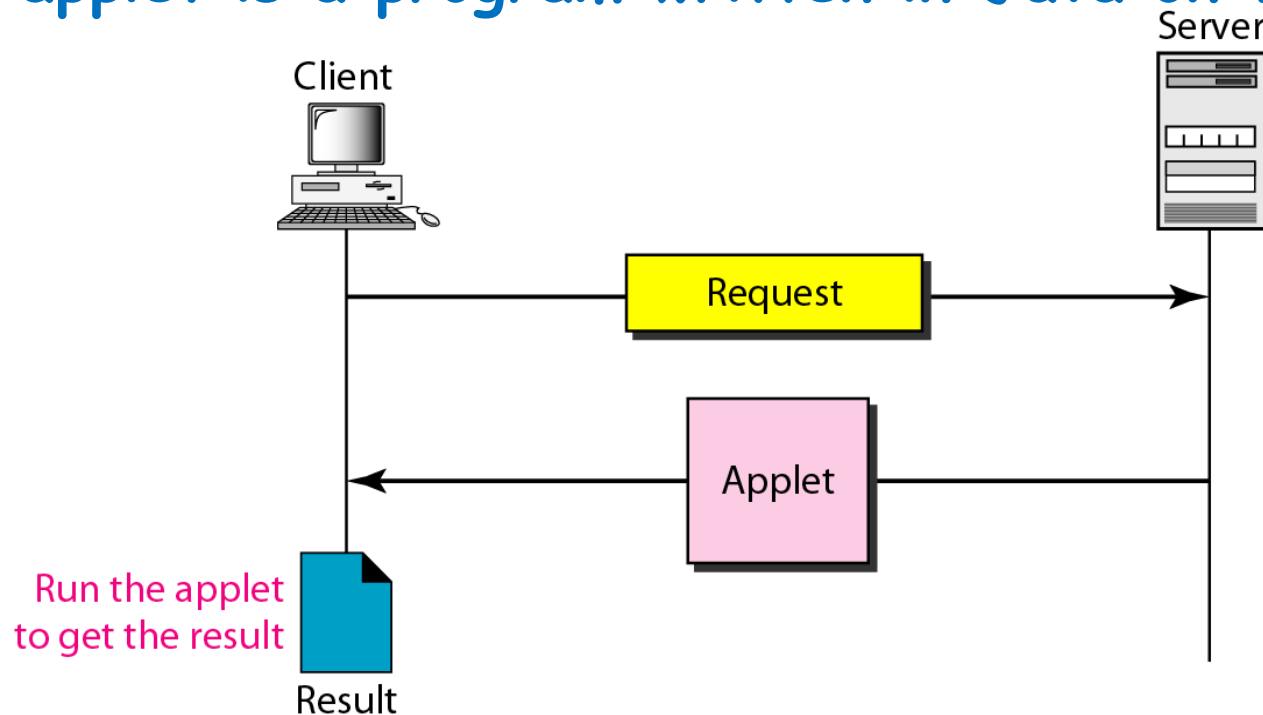
*Note*

**Dynamic documents are sometimes referred to as server-site dynamic documents.**

**Figure 27.10** *Active document using Java applet*

For many applications, we need a program or a script to be run at the client site. These are called active documents.

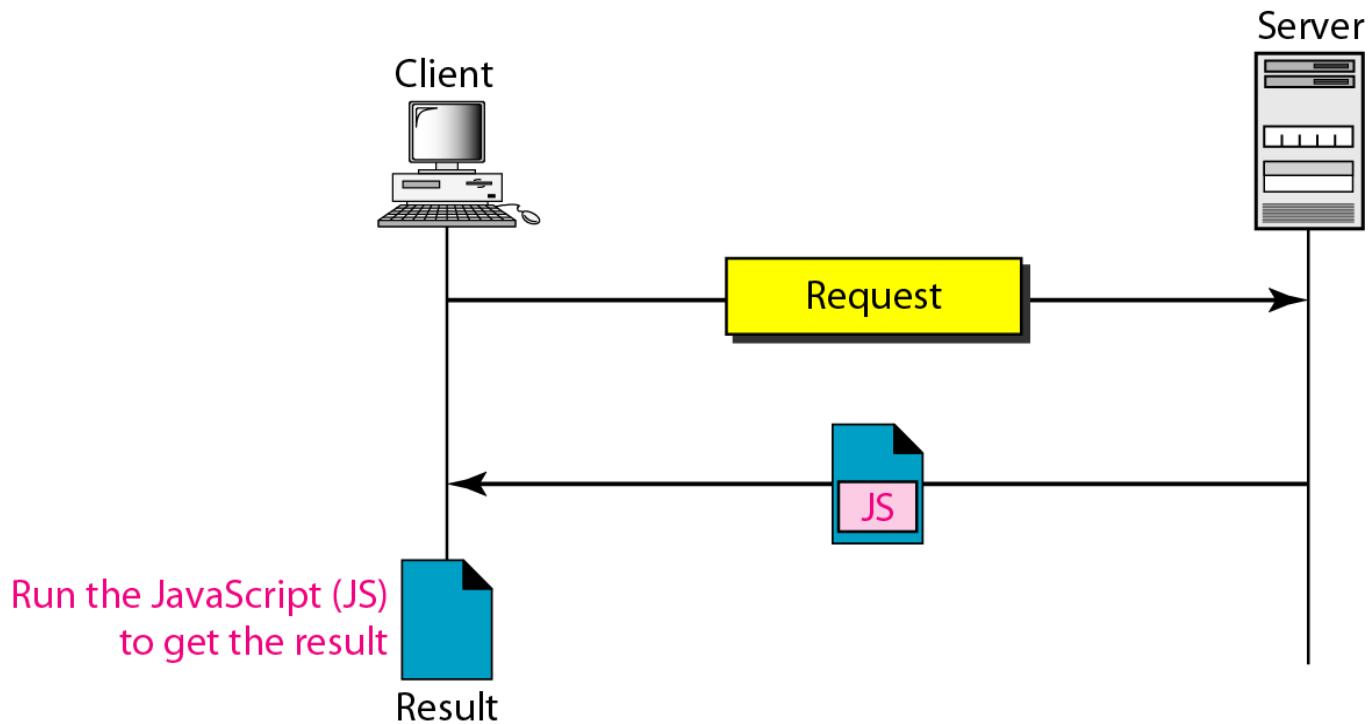
An applet is a program written in Java on the server.



---

**Figure 27.11** *Active document using client-site script*

---





*Note*

**Active documents are sometimes referred to as client-site dynamic documents.**

## 27-3 HTTP

*The Hypertext Transfer Protocol (HTTP) is a protocol used mainly to access data on the World Wide Web. HTTP functions as a combination of **FTP** and **SMTP**.*

*Topics discussed in this section:*

HTTP Transaction

Persistent Versus Nonpersistent Connection



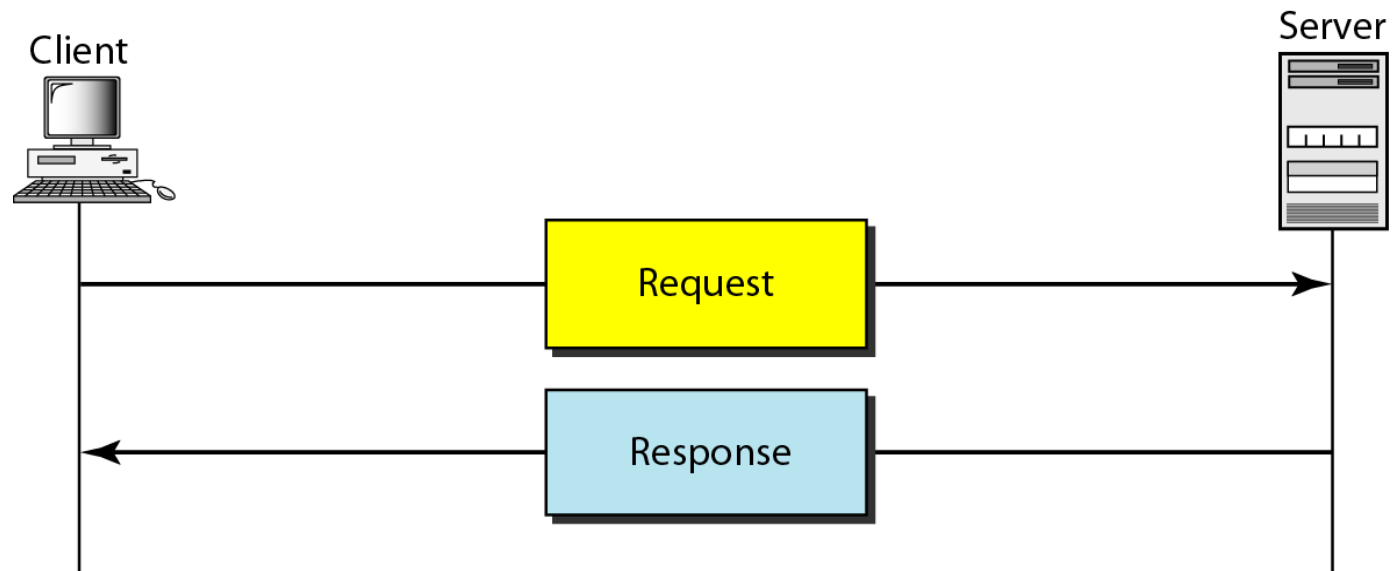
*Note*

**HTTP uses the services of TCP on well-known port 80.**

---

**Figure 27.12** *HTTP transaction*

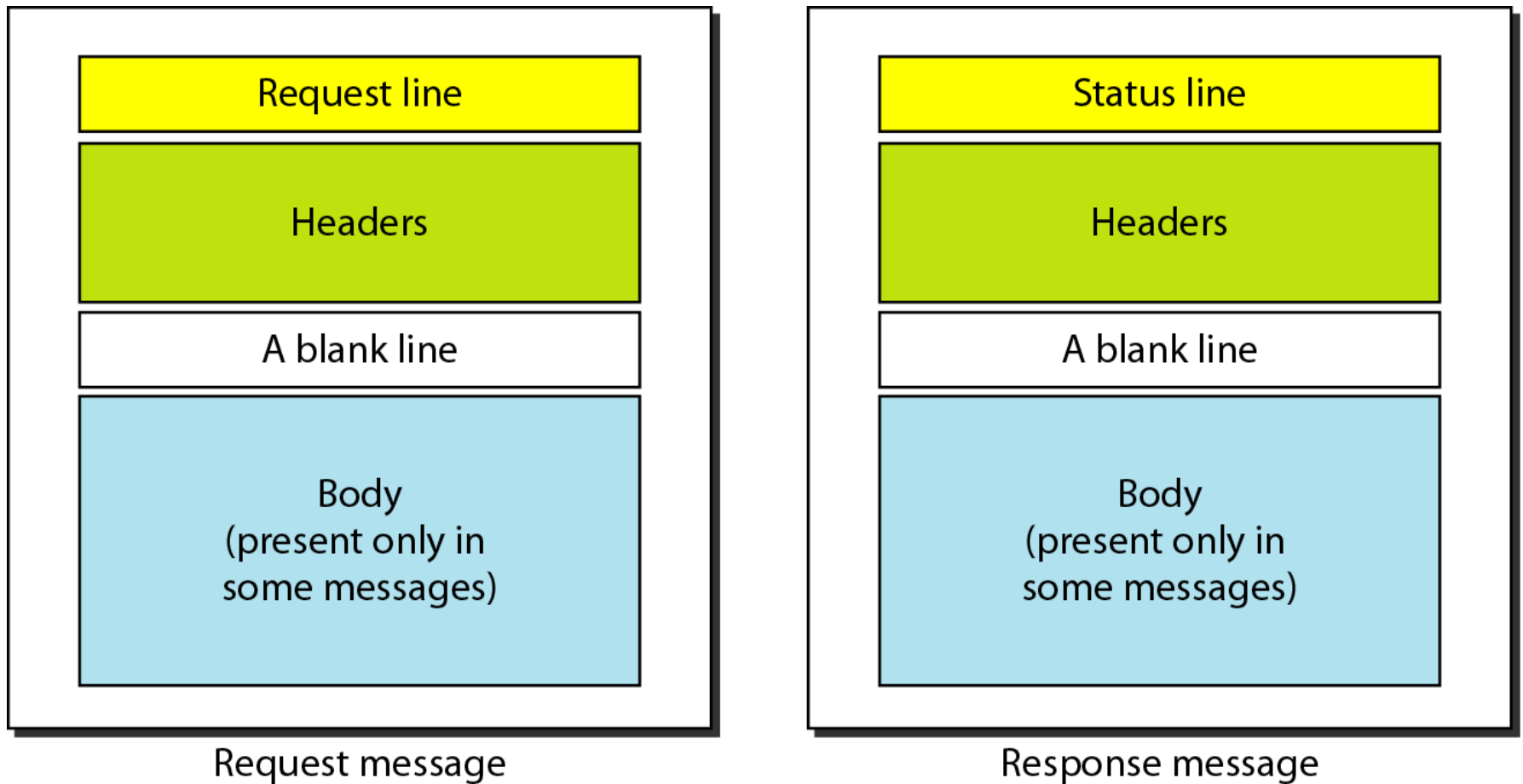
---



---

**Figure 27.13** *Request and response messages*

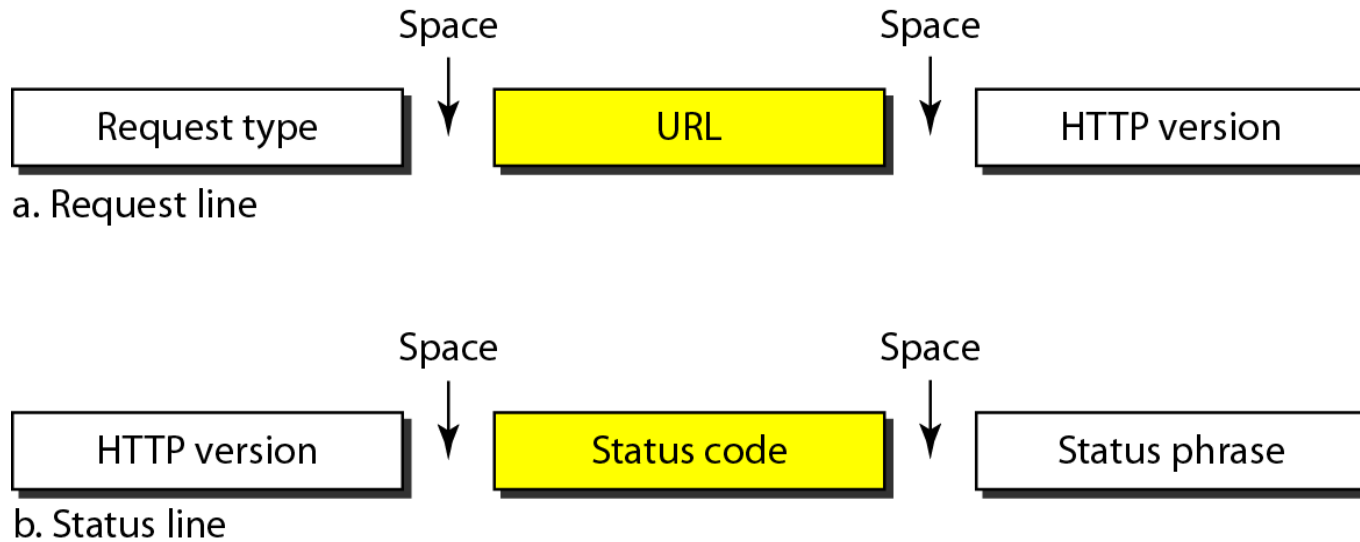
---



---

**Figure 27.14** *Request and status lines*

---



**Table 27.1** *Methods*

| <i>Method</i> | <i>Action</i>                                                     |
|---------------|-------------------------------------------------------------------|
| GET           | Requests a document from the server                               |
| HEAD          | Requests information about a document but not the document itself |
| POST          | Sends some information from the client to the server              |
| PUT           | Sends a document from the server to the client                    |
| TRACE         | Echoes the incoming request                                       |
| CONNECT       | Reserved                                                          |
| OPTION        | Inquires about available options                                  |

**Table 27.2** *Status codes*

| <i>Code</i>          | <i>Phrase</i> | <i>Description</i>                                                                               |
|----------------------|---------------|--------------------------------------------------------------------------------------------------|
| <b>Informational</b> |               |                                                                                                  |
| <b>100</b>           | Continue      | The initial part of the request has been received, and the client may continue with its request. |
| <b>101</b>           | Switching     | The server is complying with a client request to switch protocols defined in the upgrade header. |
| <b>Success</b>       |               |                                                                                                  |
| <b>200</b>           | OK            | The request is successful.                                                                       |
| <b>201</b>           | Created       | A new URL is created.                                                                            |
| <b>202</b>           | Accepted      | The request is accepted, but it is not immediately acted upon.                                   |
| <b>204</b>           | No content    | There is no content in the body.                                                                 |

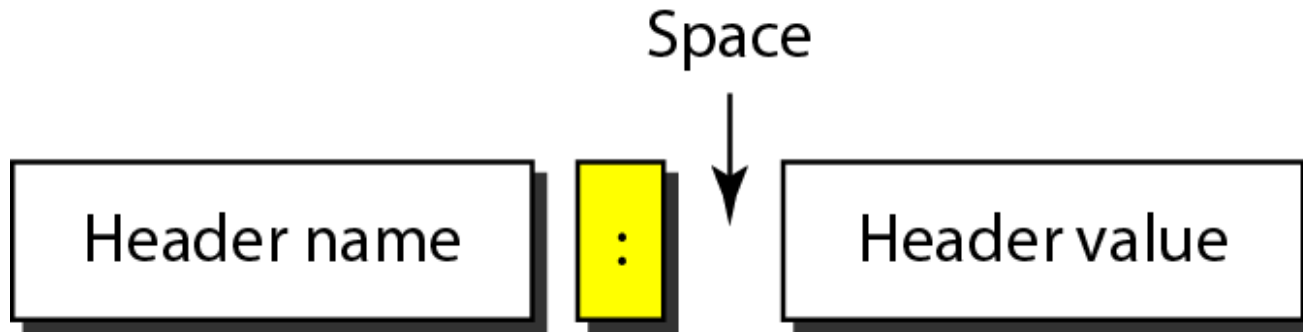
**Table 27.2** *Status codes (continued)*

| <i>Code</i>         | <i>Phrase</i>         | <i>Description</i>                                                          |
|---------------------|-----------------------|-----------------------------------------------------------------------------|
| <b>Redirection</b>  |                       |                                                                             |
| <b>301</b>          | Moved permanently     | The requested URL is no longer used by the server.                          |
| <b>302</b>          | Moved temporarily     | The requested URL has moved temporarily.                                    |
| <b>304</b>          | Not modified          | The document has not been modified.                                         |
| <b>Client Error</b> |                       |                                                                             |
| <b>400</b>          | Bad request           | There is a syntax error in the request.                                     |
| <b>401</b>          | Unauthorized          | The request lacks proper authorization.                                     |
| <b>403</b>          | Forbidden             | Service is denied.                                                          |
| <b>404</b>          | Not found             | The document is not found.                                                  |
| <b>405</b>          | Method not allowed    | The method is not supported in this URL.                                    |
| <b>406</b>          | Not acceptable        | The format requested is not acceptable.                                     |
| <b>Server Error</b> |                       |                                                                             |
| <b>500</b>          | Internal server error | There is an error, such as a crash, at the server site.                     |
| <b>501</b>          | Not implemented       | The action requested cannot be performed.                                   |
| <b>503</b>          | Service unavailable   | The service is temporarily unavailable, but may be requested in the future. |

---

**Figure 27.15** *Header format*

---



# Header format

■ **General header** The general header gives general information about the message and can be present in both a request and a response.

■ **Request header** The request header can be present only in a request message. It specifies the client's configuration and the client's preferred document format.

■ **Response header** The response header can be present only in a response message. It specifies the server's configuration and special information about the request.

■ **Entity header** The entity header gives information about the body of the document.

**Table 27.3** *General headers*

| <i>Header</i> | <i>Description</i>                                   |
|---------------|------------------------------------------------------|
| Cache-control | Specifies information about caching                  |
| Connection    | Shows whether the connection should be closed or not |
| Date          | Shows the current date                               |
| MIME-version  | Shows the MIME version used                          |
| Upgrade       | Specifies the preferred communication protocol       |

**Table 27.4** *Request headers*

| <i>Header</i>       | <i>Description</i>                                     |
|---------------------|--------------------------------------------------------|
| Accept              | Shows the medium format the client can accept          |
| Accept-charset      | Shows the character set the client can handle          |
| Accept-encoding     | Shows the encoding scheme the client can handle        |
| Accept-language     | Shows the language the client can accept               |
| Authorization       | Shows what permissions the client has                  |
| From                | Shows the e-mail address of the user                   |
| Host                | Shows the host and port number of the server           |
| If-modified-since   | Sends the document if newer than specified date        |
| If-match            | Sends the document only if it matches given tag        |
| If-non-match        | Sends the document only if it does not match given tag |
| If-range            | Sends only the portion of the document that is missing |
| If-unmodified-since | Sends the document if not changed since specified date |
| Referrer            | Specifies the URL of the linked document               |
| User-agent          | Identifies the client program                          |

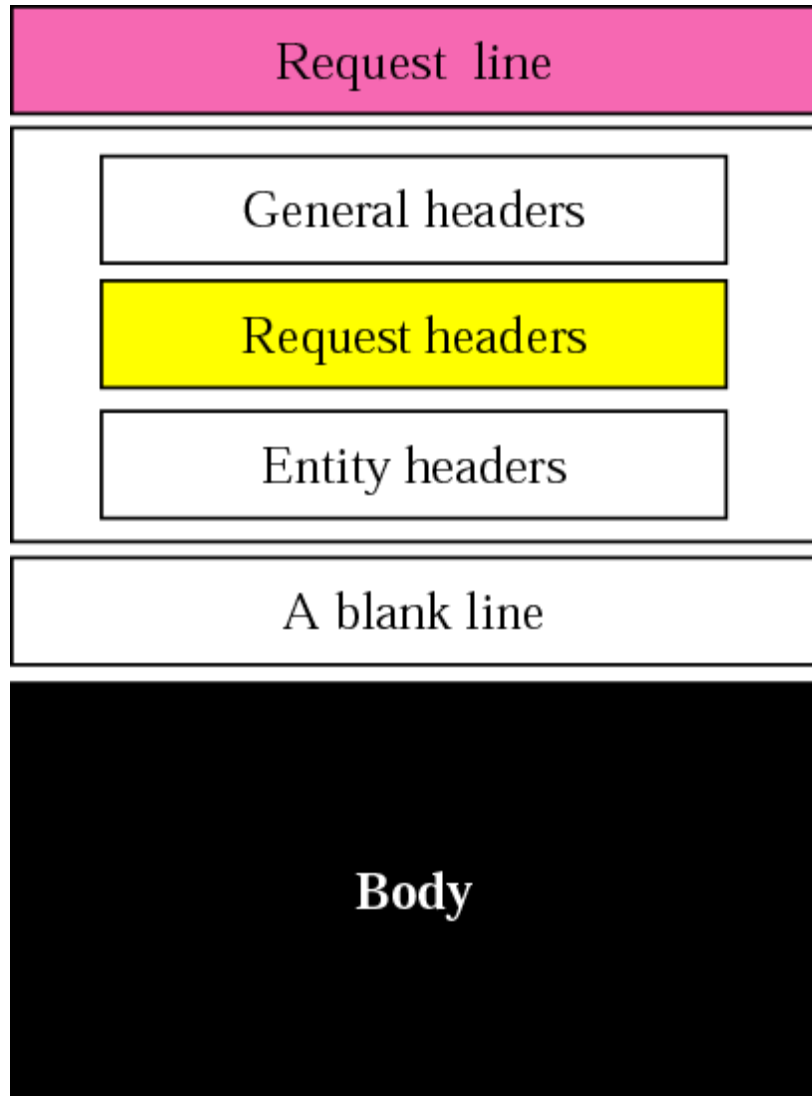
**Table 27.5** *Response headers*

| <i>Header</i> | <i>Description</i>                                     |
|---------------|--------------------------------------------------------|
| Accept-range  | Shows if server accepts the range requested by client  |
| Age           | Shows the age of the document                          |
| Public        | Shows the supported list of methods                    |
| Retry-after   | Specifies the date after which the server is available |
| Server        | Shows the server name and version number               |

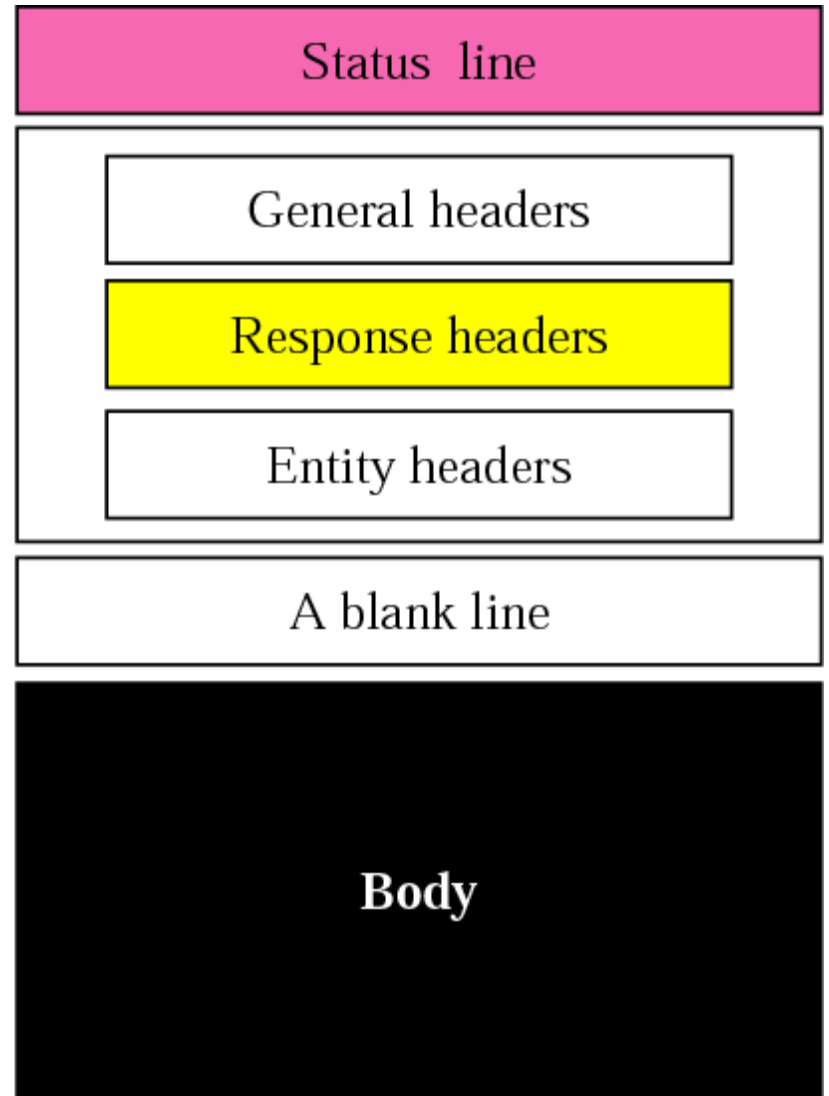
**Table 27.6** *Entity headers*

| <i>Header</i>    | <i>Description</i>                                      |
|------------------|---------------------------------------------------------|
| Allow            | Lists valid methods that can be used with a URL         |
| Content-encoding | Specifies the encoding scheme                           |
| Content-language | Specifies the language                                  |
| Content-length   | Shows the length of the document                        |
| Content-range    | Specifies the range of the document                     |
| Content-type     | Specifies the medium type                               |
| Etag             | Gives an entity tag                                     |
| Expires          | Gives the date and time when contents may change        |
| Last-modified    | Gives the date and time of the last change              |
| Location         | Specifies the location of the created or moved document |

# Headers



Request message



Response message



## Example 27.1

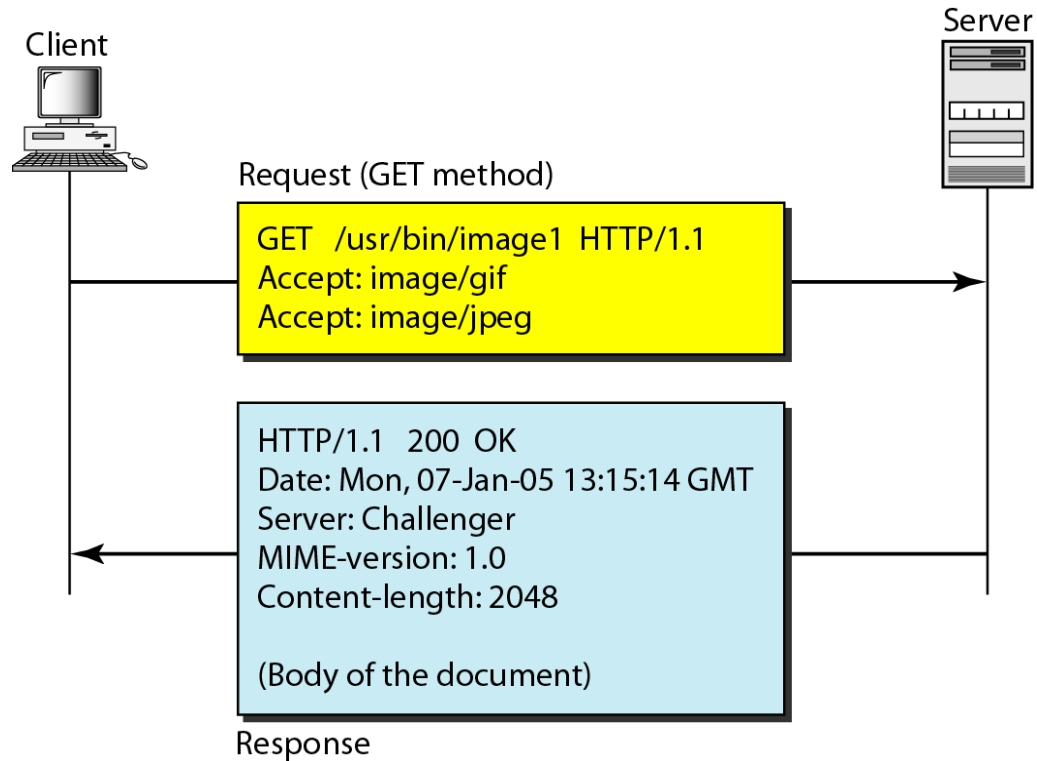
---

*This example retrieves a document. We use the GET method to retrieve an image with the path /usr/bin/image1. The request line shows the method (GET), the URL, and the HTTP version (1.1). The header has two lines that show that the client can accept images in the GIF or JPEG format. The request does not have a body. The response message contains the status line and four lines of header. The header lines define the date, server, MIME version, and length of the document. The body of the document follows the header (see Figure 27.16).*

---

**Figure 27.16** *Example 27.1*

---

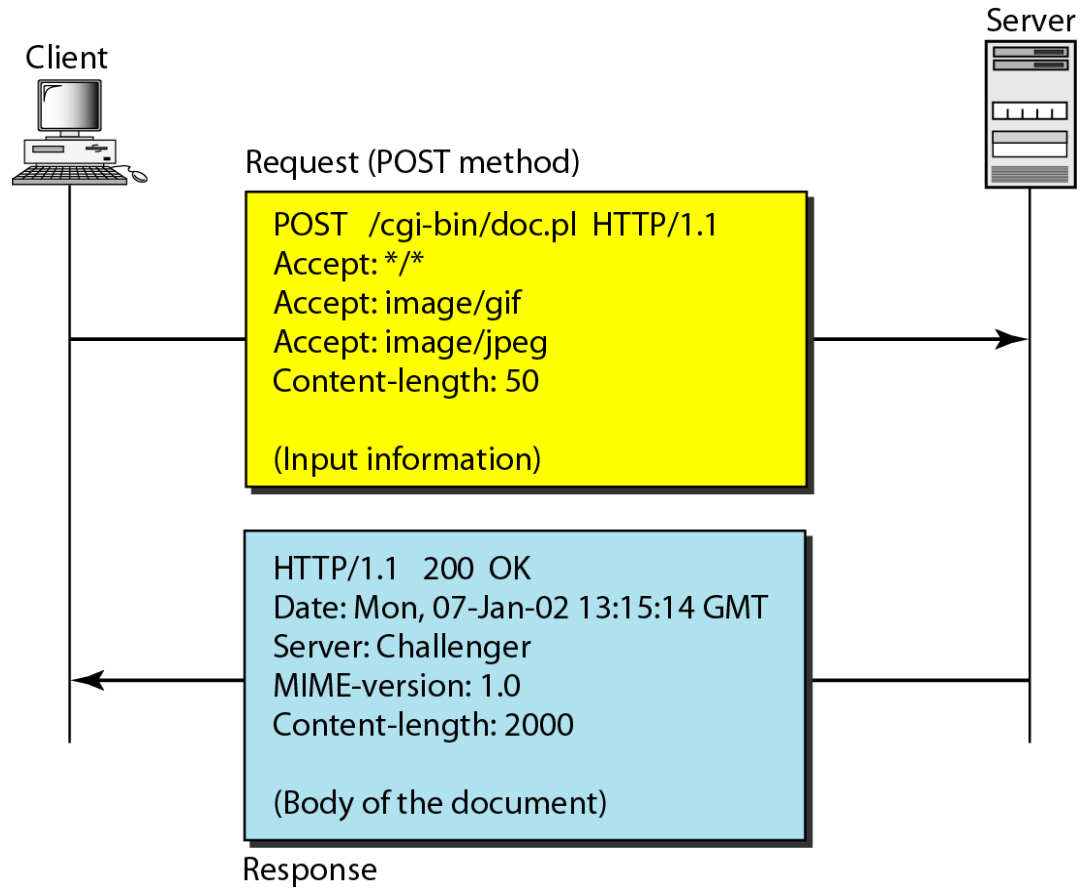




## Example 27.2

*In this example, the client wants to send data to the server. We use the POST method. The request line shows the method (POST), URL, and HTTP version (1.1). There are four lines of headers. The request body contains the input information. The response message contains the status line and four lines of headers. The created document, which is a CGI document, is included as the body (see Figure 27.17).*

**Figure 27.17** *Example 27.2*





## Example 27.3

*HTTP uses ASCII characters. A client can directly connect to a server using TELNET, which logs into port 80 (see next slide). The next three lines show that the connection is successful. We then type three lines. The first shows the request line (GET method), the second is the header (defining the host), the third is a blank, terminating the request. The server response is seven lines starting with the status line. The blank line at the end terminates the server response. The file of 14,230 lines is received after the blank line (not shown here). The last line is the output by the client.*

## *Example 27.3 (continued)*

```
$ telnet www.mhhe.com 80
```

```
Trying 198.45.24.104 ...
```

```
Connected to www.mhhe.com (198.45.24.104).
```

```
Escape character is '^]'.
```

```
GET /engcs/compsci/forouzan HTTP/1.1
```

```
From: forouzanbehrouz@fhda.edu
```

```
HTTP/1.1 200 OK
```

```
Date: Thu, 28 Oct 2004 16:27:46 GMT
```

```
Server: Apache/1.3.9 (Unix) ApacheJServ/1.1.2 PHP/4.1.2 PHP/3.0.18
```

```
MIME-version:1.0
```

```
Content-Type: text/html
```

## ***Nonpersistent Connection***

In a nonpersistent connection, one TCP connection is made for each request/response. The following lists the steps in this strategy:

1. The client opens a TCP connection and sends a request.
2. The server sends the response and closes the connection.
3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

## ***Persistent Connection***

HTTP version 1.1 specifies a persistent connection by default. In a persistent connection, the server leaves the connection open for more requests after sending a response. The server can close the connection at the request of a client or if a time-out has been reached.



*Note*

**HTTP version 1.1 specifies a persistent connection by default.**

# Questions